Container Breakout Attempt response

Cloud · **Critical** severity · SOC IR Playbook

Incident type	Container Runtime Security - Escape Attempt
Severity	Critical
Priority	Critical
Detection sources	Runtime Security Tools, SIEM, EDR, Kubernetes Audit Logs, Falco Rules, Container Logs

Scenario

An attacker gains access to a container and attempts to escape the isolated environment to interact with the host operating system, escalate privileges or compromise other containers, pods or underlying infrastructure.

MITRE ATT&CK

T1059, T1203, T1611

Preparation

Implement runtime security Use tools like Falco, Aqua, Sysdig or Wiz Runtime for real-time detection Enable Kubernetes and Docker audit logging Capture container activity and host-level access attempts Harden container images Use minimal base images, scan for vulnerabilities and remove unnecessary tools (e.g., curl, bash) Apply Pod Security Policies / OPA Prevent privilege escalation, host mounts and container privilege mode Monitor inter-container traffic Enable east-west container traffic monitoring using NDR or eBPF-based tools

Detection & Analysis

Alert triggered Attempt to access host filesystem (/proc, /root), escalate privileges or spawn unexpected binaries Review container activity Check for nsenter, chroot, mount, apt, wget or suspicious execs Identify affected container and node Determine pod name, namespace and underlying host VM/node Analyse attacker behaviour Was this a misconfiguration exploit (e.g., privileged: true) or remote code execution?

Containment

Stop compromised pod or container Forcefully delete or isolate the instance immediately Isolate affected node Remove the node from cluster scheduling and limit communication Suspend service account access Especially if the container had access to Kubernetes API or secrets Snapshot affected container If forensics is required, preserve memory and logs where possible

Eradication

Investigate the root cause Vulnerable image, over-permissive configuration or exposed interface Patch vulnerable workloads Rebuild and redeploy affected pods with fixed configuration or image Rotate secrets and credentials Especially if stored in environment variables, configMaps or volumes Remove backdoors or malicious tools Search for rogue binaries, cron jobs or injected scripts in containers or host

Recovery

Rebuild workloads from trusted images Use CI/CD pipelines with image signing and scanning Reinstate node after sanitisation Only after full forensic validation of the host system Resume services Reintroduce pods gradually and monitor closely for recurrence Enhance monitoring for affected namespace or deployment Apply anomaly detection for future deviations

Lessons Learned & Reporting

Conduct a post-mortem analysis Document breakout vector, timeline and exposed assets Improve container security policies Enforce strict resource isolation and prevent reuse of affected patterns Educate DevOps teams On secure container configuration, minimal permissions and runtime risks Report if required If data exposure or

host compromise occurred, notify regulatory bodies or clients Update runbooks and response workflows Add new rules and controls based on this attack scenario

Tools typically involved

Runtime Security (e.g., Falco, Sysdig Secure, Aqua, Prisma Cloud Compute, Wiz); Kubernetes Audit Logs and RBAC logs; SIEM (e.g., Sentinel, Splunk, QRadar); EDR (for host nodes, e.g., CrowdStrike, Cortex XDR); NDR (for container traffic visibility); Image Scanning (e.g., Trivy, Clair, Anchore)

Success metrics (SLA targets)

Detection Time <5 minutes from breakout attempt Pod Termination Time <10 minutes from alert Root Cause Fix Time <48 hours Node Revalidation Completion Within 24 hours post-removal Image Hardening Review 100% of similar deployments audited within 3 business days